PW-STR-017 · DOCUMENTO DE INGENIERIA

PitWall — Estructura del proyecto

Árbol del repositorio con explicación de cada carpeta

VERSION DEL DOCUMENTO **1.4**	ESTADO **Vigente**
FECHA **2026-09-22**	AUTOR **Luis Miguel**

Documento generado automaticamente a partir de `17-PROJECT-STRUCTURE.md`. El fichero Markdown es la unica fuente de verdad; este PDF es un artefacto derivado y no debe editarse por separado.

PitWall es un proyecto no oficial sin afiliacion con Electronic Arts Inc., Codemasters, la FIA ni Formula One World Championship Limited. Las marcas citadas pertenecen a sus respectivos titulares y se emplean unicamente de forma descriptiva.

Indice

1. Principios de organización

2. Árbol completo

3. Explicación por carpeta

- 3.1. Raíz
- 3.2. config/
- 3.3. core/src/ — el corazón
- 3.4. core/tools/
- 3.5. ui/src/
- 3.6. tests/ compartido
- 3.7. tools/ del repositorio
- 3.8. spec/ — no versionado

4. Convenios de nombres

5. Datos en tiempo de ejecución

6. Qué NO existe, y por qué

Estructura del proyecto

1. Principios de organización

#	Principio
E1	**La estructura refleja la arquitectura.** Una carpeta por módulo de `04-ARCHITECTURE.md`
E2	**Las fronteras son visibles.** Que `pw::domain` no dependa de `pw::proto` debe verse en el árbol
E3	**Lo generado no se versiona**, salvo que la integración continua lo verifique
E4	**Lo que no es nuestro no se versiona.** La especificación de EA vive en `spec/`, ignorada
E5	**Las pruebas viven junto a lo que prueban**, salvo fixtures y escenarios, que son compartidos

2. Árbol completo

```
pitwall/
├── .github/workflows/           integración continua
├── .gitignore
├── CMakeLists.txt               raíz de la construcción
├── CMakePresets.json            debug, release, asan, tsan, fuzz
├── LICENSE
├── README.md                    qué es, cómo compilar, aviso legal
├── vcpkg.json                   manifiesto de dependencias con versiones fijadas
│
├── config/
│   ├── default.toml             valores por defecto documentados
│   ├── dev.toml                 desarrollo: logs verbosos, rutas locales
│   └── example.toml             plantilla comentada para el usuario
│
├── core/                        ── NÚCLEO C++20/23 ──
│   ├── CMakeLists.txt
│   ├── include/pw/              cabeceras públicas, espejo de src/
│   ├── src/
│   │   ├── core/                pw::core   — Result, Channel, tipos fuertes, tiempo
│   │   ├── obs/                 pw::obs    — métricas y logging
│   │   ├── config/              pw::config
│   │   ├── mem/                 pw::mem    — BufferPool, SpscRing
│   │   ├── net/                 pw::net    — UdpReceiver, ReplaySource, sockets
│   │   ├── proto/               pw::proto  — SÓLO structs y static_assert
│   │   │   ├── header.hpp
│   │   │   ├── f1_2025/         packets.hpp, constants.hpp, catalogs.cpp
│   │   │   └── f1_2026/         packets.hpp, constants.hpp, catalogs.cpp
│   │   ├── decoder/             pw::decoder — interfaz, registro, un decoder por formato
│   │   ├── domain/              pw::domain  — modelo normalizado, SIN dependencias
│   │   ├── normalize/           pw::normalize — el único puente proto ↔ domain
│   │   ├── state/               pw::state   — motor de estado, ramas, bus
│   │   ├── processors/          pw::processors — live, lap, engineer (F3)
│   │   ├── recorder/            pw::recorder — formato .f1raw, escritor, lector
│   │   ├── persistence/         pw::persistence — catálogo, series, retención
│   │   ├── transport/           pw::transport — transporte interno, serialización
│   │   ├── shell/               pw::shell — ventana Win32 + WebView2 (SÓLO Windows)
│   │   │   ├── window.cpp           ventana, bucle de mensajes, PPP
│   │   │   ├── webview_host.cpp     entorno y control WebView2
│   │   │   ├── virtual_host.cpp     mapeo de host virtual a la carpeta de recursos
│   │   │   ├── bootstrap.cpp        inyección de endpoint efímero y token
│   │   │   ├── host_bridge.cpp      diálogos nativos, enlaces externos
│   │   │   ├── single_instance.cpp  mutex y activación de ventana
│   │   │   └── runtime_check.cpp    presencia del WebView2 Runtime
│   │   └── app/                 composición, arranque, hilos
│   ├── tests/
│   │   ├── unit/                una carpeta por módulo
│   │   ├── integration/         replay → estado
│   │   ├── fuzz/                objetivos de fuzzing
```

```
│   │   └── perf/                pruebas de rendimiento
│   ├── apps/
│   │   ├── PitWall/             EJECUTABLE DE ESCRITORIO (shell + núcleo)
│   │   └── pitwall-cli/         mismo núcleo SIN interfaz: pruebas, lotes, CI en Linux
│   └── tools/                   ejecutables auxiliares
│       ├── common/              infraestructura compartida por las herramientas
│       │   └── stop_signal*.{hpp,cpp}  adaptadores de Ctrl+C / SIGINT (F1.1-13)
│       ├── pw-listen/           escucha e imprime estadísticas (F1.1-13)
│       │   ├── main.cpp                argumentos, señal y código de salida
│       │   └── listen_app.hpp/.cpp     la lógica, probada sin lanzar el proceso
│       ├── pw-load/             emisor de carga UDP sostenida (F1.1-14)
│       │   ├── main.cpp                argumentos, señal y código de salida
│       │   └── load_generator.hpp/.cpp ritmo por fecha límite; mide lo conseguido
│       ├── pw-capture/          info, verify, extract, slice, anonymize, replay…
│       └── pw-gen/              generador sintético de casos degradados
│
├── ui/                          ── FRONTEND ──
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── index.html
│   ├── public/
│   ├── src/
│   │   ├── app/                 arranque, enrutado, disposición
│   │   ├── net/                 cliente del transporte interno, decodificadores, puente con el shell
│   │   ├── stores/              connection, session, timing, telemetry, event, view
│   │   ├── screens/             una carpeta por pantalla
│   │   │   ├── Status/ Diagnostics/ Timing/  Telemetry/  TrackMap/
│   │   │   ├── Tyres/ Damage/  DriverDetail/  Compare/
│   │   │   └── History/ Replay/ Settings/ Strategy/
│   │   ├── components/
│   │   │   ├── primitives/      panel, pestaña, botón, indicador de estado
│   │   │   ├── data/            ValueWithProvenance, TabularNumber, DataAge
│   │   │   ├── charts/          LineChart, DeltaChart, decimado
│   │   │   ├── track/           lienzo del mapa, capas, proyección
│   │   │   └── timing/          fila, torre, bloque de sectores
│   │   ├── domain/
│   │   │   ├── generated.ts     GENERADO desde C++, no editar
│   │   │   └── index.ts
│   │   ├── design/              tokens, temas, tipografía
│   │   └── utils/
│   └── tests/
│       ├── unit/
│       └── e2e/
│
├── tests/                       ── COMPARTIDO ──
│   ├── fixtures/packets/
│   │   ├── 2025/                16 .bin + 16 .json de valores esperados
│   │   └── 2026/                17 .bin + 17 .json
│   ├── fixtures/events/         un fixture por código de evento
│   ├── scenarios/               capturas .f1raw recortadas y anonimizadas
```

```
│   └── corpus/                  corpus semilla de fuzzing
│
├── tools/                       ── UTILIDADES DEL REPOSITORIO ──
│   ├── package.json
│   ├── md2pdf.mjs               genera PDF desde Markdown
│   ├── build-all-pdfs.mjs
│   ├── gen_ts_types.py          C++ domain → TypeScript
│   ├── traceability.py          requisito ↔ prueba
│   ├── check_deps.py            verificación del grafo de módulos
│   ├── coverage.py
│   └── dev.py                   arranca núcleo + frontend
│
├── docs/                        ── DOCUMENTACIÓN ──
│   ├── 01-REQUIREMENTS.md  + .pdf
│   ├── 02-RESEARCH.md
│   ├── 03-UDP-PROTOCOL.md
│   ├── 04-ARCHITECTURE.md
│   ├── 05-DATA-MODEL.md
│   ├── 06-REALTIME-PIPELINE.md
│   ├── 07-RECORDING-AND-REPLAY.md
│   ├── 08-FRONTEND-PLAN.md
│   ├── 09-BACKEND-PLAN.md
│   ├── 10-TESTING-PLAN.md
│   ├── 11-IMPLEMENTATION-PLAN.md
│   ├── 12-MVP.md
│   ├── 13-BACKLOG.md
│   ├── 14-ROADMAP.md
│   ├── 15-ADR.md
│   ├── 16-DEVELOPMENT-GUIDE.md
│   ├── 17-PROJECT-STRUCTURE.md
│   ├── 18-GLOSSARY.md
│   └── assets/
│
├── spec/                        ── NO VERSIONADO ──
│   └── (especificación oficial de EA descargada del hilo oficial)
│
└── build/                       ── NO VERSIONADO ──
```

3. Explicación por carpeta

3.1. Raíz

Elemento	Contenido	Notas
`CMakePresets.json`	Los cinco presets	Permite `cmake --preset debug` sin recordar banderas
`vcpkg.json`	Dependencias con **versiones fijadas**	Reproducibilidad (RNF-092)
`README.md`	Qué es, cómo compilar, **aviso legal**	El aviso de `01-REQUIREMENTS.md` §26.3 va aquí
`.gitignore`	Excluye `spec/`, `build/`, `data/`, `node_modules/`	E3, E4

3.2. `config/`

Tres ficheros con propósitos distintos: `default.toml` documenta cada opción y su valor por defecto; `dev.toml` configura el desarrollo; `example.toml` es la plantilla que se entrega al usuario.

3.3. `core/src/` — el corazón

El orden de las carpetas **no es alfabético**: sigue el flujo de dependencias, de lo más básico a lo más compuesto.

Carpeta	Regla que la define
`core/`	No depende de nada del proyecto
`obs/`, `config/`, `mem/`	Sólo dependen de `core`
`proto/`	**Sólo structs, constantes y `static_assert`.** Ninguna lógica. No conoce `domain`
`decoder/`	Conoce `proto`. **No conoce `domain`**
`domain/`	**No depende de nada.** Ni siquiera de `proto`
`normalize/`	**El único** que incluye a la vez `proto` y `domain`
`state/`	Conoce `domain`. **No conoce `proto`**
`processors/`, `recorder/`, `persistence/`, `transport/`	Consumen `state` y `domain`
`shell/`	**El único módulo específico de Windows.** Depende de `app`; **nadie depende de él**. Quitarlo deja un núcleo que compila y se ejecuta sin interfaz
`app/`	Lo compone todo. El único con visión global

`tools/check_deps.py` verifica estas reglas en cada cambio (E2, RNF-084), incluida la más importante tras el cambio a aplicación de escritorio: las APIs específicas de plataforma sólo aparecen en **adaptadores de plataforma explícitos y controlados**. `pw::shell` es el propietario general de la API Win32 de interfaz; fuera de él sólo hay excepciones estrechas para adaptadores de bajo nivel —los sockets y la prioridad de T1, ambos en `pw::net`—, declaradas fichero a fichero y cabecera a cabecera en `tools/check_deps.py`. Es lo que mantiene el núcleo compilando en Linux y lo que hace que el shell sea sustituible si algún día hiciera falta otra cosa.

Por qué `proto/` tiene un subdirectorio por formato

```
proto/
├── header.hpp              común a ambos: 29 bytes estables
├── f1_2025/
│   ├── constants.hpp       kMaxCars = 22
│   ├── packets.hpp         16 structs + static_assert
│   └── catalogs.cpp        equipos, pilotos, circuitos de 2025
└── f1_2026/
    ├── constants.hpp       kMaxCars = 24
    ├── packets.hpp         17 structs + static_assert
    └── catalogs.cpp        equipos (485 Audi, 486 Cadillac), circuitos (42 Madrid)
```

Añadir un formato 2027 es **añadir una carpeta**. Ninguna otra se toca (RNF-040). La estructura hace visible la propiedad que el diseño promete.

3.4. `core/tools/`

Ejecutables auxiliares, separados del binario principal para poder usarse en integración continua y en depuración sin arrancar la interfaz.

Herramienta	Cuándo se usa
`pw-listen`	Primera prueba contra el juego real
`pw-load`	Carga sintética sostenida para RNF-001; no habla el protocolo
`pw-capture`	A diario: verificar, recortar, anonimizar, extraer fixtures, reproducir
`pw-gen`	Generar los casos degradados que no se pueden capturar

3.5. `ui/src/`

Carpeta	Regla
`net/`	Único punto que habla con el núcleo y con el shell anfitrión
`stores/`	Estado. `telemetry` **mantiene búferes circulares fuera de React**
`screens/`	Una carpeta por pantalla, con sus componentes privados
`components/`	Sólo lo reutilizable. Un componente usado una vez vive en su pantalla
`domain/generated.ts`	**Generado desde C++.** La integración continua falla si difiere
`design/`	Tokens y temas. Ningún color literal fuera de aquí

3.6. `tests/` compartido

Fixtures y escenarios son compartidos porque los usan tanto el núcleo como las pruebas extremo a extremo.

Carpeta	Contenido	Tamaño típico
`fixtures/packets/`	Un `.bin` y un `.json` por paquete y formato	< 2 KB
`fixtures/events/`	Uno por código de evento	< 100 B
`scenarios/`	Capturas recortadas y **anonimizadas**	100 KB – 2 MB
`corpus/`	Semillas de fuzzing	variable

Regla absoluta: toda captura versionada está anonimizada (TST-03, PRV-03).

Las capturas grandes de rendimiento **no se versionan**: viven fuera y se descargan.

3.7. `tools/` del repositorio

Script	Qué garantiza
`md2pdf.mjs`	Que el PDF y el Markdown nunca divergen
`gen_ts_types.py`	Que los tipos de C++ y TypeScript nunca divergen
`traceability.py`	Que ningún requisito `MUST` quede sin prueba
`check_deps.py`	Que el grafo de módulos no se degrade
`dev.py`	Que arrancar el entorno completo sea una orden

Los cinco existen porque automatizan **propiedades que se pierden solas** cuando hay prisa.

3.8. `spec/` — no versionado

Contiene la especificación oficial de EA descargada del hilo oficial. **Nunca se versiona.**

El motivo es explícito en el propio documento de EA: *«This information is being provided under license from EA for reference purposes only»*. Redistribuirla en un repositorio público excedería esa licencia (`01-REQUIREMENTS.md` §26.3).

El `README` explica cómo obtenerla y dónde colocarla.

---

4. Convenios de nombres

Elemento	Convenio	Ejemplo
Espacios de nombres C++	`snake_case` bajo `pw`	`pw::proto::f1_2026`
Tipos C++	`PascalCase`	`SessionStateEngine`
Funciones y variables C++	`camelCase`	`monotonicNowNs()`
Miembros privados C++	`camelCase_` con guion bajo final	`lastFrameId_`
Constantes C++	`kPascalCase`	`kMaxCars`
Campos transcritos del protocolo	**Se conserva el nombre oficial** `m_xxx`	`m_packetFormat`
Ficheros C++	`snake_case`	`session_state_engine.cpp`
Componentes React	`PascalCase`	`TimingTower.tsx`
Hooks y utilidades TS	`camelCase`	`useTelemetryChannel.ts`
Documentos	`NN-NOMBRE-EN-MAYUSCULAS.md`	`03-UDP-PROTOCOL.md`
Fixtures	`NN_nombre_paquete.bin`	`16_car_telemetry2.bin`
Escenarios	`<formato>_<escenario>.f1raw`	`2026_flashback_simple.f1raw`

> **La excepción del `m_` es deliberada.** En `pw::proto` los campos conservan literalmente el nombre de la especificación oficial, aunque contradiga el estilo del proyecto. La trazabilidad campo a campo con el documento de EA vale más que la coherencia estilística, y el prefijo actúa además como señal visual de que se está en la capa de protocolo. En `pw::domain` el prefijo está **prohibido**, y esa asimetría es lo que hace evidente de un vistazo en qué capa se está leyendo.

---

5. Datos en tiempo de ejecución

Fuera del repositorio, en `%LOCALAPPDATA%\PitWall\data\`:

```
data/
├── captures/          .f1raw + índices
├── catalog.db         catálogo
├── series/            columnar particionado por sesión y vehículo
├── derived/           trazados, vueltas remuestreadas, métricas cacheadas
└── logs/
```

Separar los datos del repositorio evita el accidente clásico de comprometer gigabytes de capturas, y permite borrar `build/` sin perder sesiones.

---

6. Qué NO existe, y por qué

Ausencia	Motivo
`src/utils/` genérico en el núcleo	Se convierte en el vertedero del proyecto. Cada utilidad vive en su módulo
Carpeta `common/` o `shared/`	Es `pw::core`, con reglas explícitas
`interfaces/` separada de las implementaciones	Una interfaz vive junto a lo que abstrae
`plugins/`	Sin carga dinámica (`04-ARCHITECTURE.md` §8)
`scripts/` en la raíz	Están en `tools/`, con propósito documentado
`third_party/` con código copiado	Las dependencias van por manifiesto, con versión y licencia verificadas
`assets/` con recursos del juego	No se distribuye ningún activo de EA ni de la F1
`docker/` o `Dockerfile` para el producto	Docker ni ejecuta ni distribuye PitWall (ADR-016). Si algún día aparece uno, será exclusivamente para integración continua y vivirá bajo `.github/`
`server/` o `web/`	No hay producto web. La interfaz vive en `ui/` y se aloja en el WebView2 del shell (ADR-002 v2)

Fin de `PW-STR-017` v1.1